

Project Development

PocketSmart AI – Your Smart Budget & Recommendation Assistant

1. Development Overview

PocketSmart AI is developed as a modular AI-powered web application using Python and FastAPI.

The system connects the frontend interface with Gemini 1.5 Flash Pro to generate personalized recommendations.

2. Backend Development

FastAPI is used to implement the backend API layer.

The backend manages:

- Routing
 - User authentication
 - Session management
 - Recommendation requests
 - AI communication
 - Response handling
-

3. Main API Routes

Home Planner

`/generate-home`

Generates home interior recommendations based on user requirements and budget.

Party Planner

`/generate-party`

Generates recommendations for party/event planning.

Jewelry Planner

`/generate-jewelry`

Accepts text and optional image inputs to generate jewelry recommendations.

4. Authentication Development

The project includes:

`/register`
`/login`
`/logout`
`/token`

These routes support registration, authentication and authorized access.

JWT tokens are used for secure authentication.

5. Session Development

The project includes:

`/session-info`
`/session-data`

These endpoints support user-specific session information and personalization.

6. Recommendation Development

The recommendation process follows:

```
graph TD
    A[User Input] --> B[Budget & Preferences]
    B --> C[FastAPI Request]
    C --> D[Recommendation Logic]
    D --> E[Gemini AI]
    E --> F[AI Response]
    F --> G[Formatted Recommendation]
```

7. Frontend Development

The frontend uses:

- HTML
- CSS
- JavaScript
- Jinja2

The interface includes dedicated forms for each planner.

AI results are displayed using structured layouts containing recommendation details such as products, services and prices.

8. Gemini AI Integration

Gemini 1.5 Flash Pro is used for:

- Budget interpretation
- Context understanding
- Recommendation generation
- Image analysis
- Personalized suggestions

The project uses prompt-based AI processing to generate domain-specific results.

9. External Platform Integration

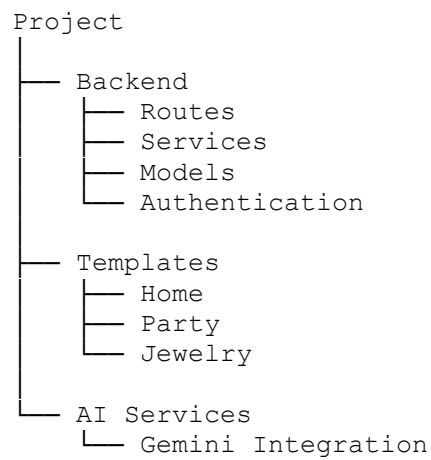
The recommendation system is designed around product/service information from platforms including:

- Amazon
- Flipkart
- IKEA
- Swiggy
- Zomato
- OYO

Mock API calls or simulated scraping can be used where required.

10. Development Structure

A modular architecture is followed to keep the project maintainable.



11. Server Execution

The FastAPI application is designed to run using Uvicorn.

Example:

```
uvicorn main:app --reload
```

The exact command may vary according to the project's actual entry-point file.